SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 9 – Implement and Test the Billing Program

Aim

To implement the complete program (tariff.h + billing.c) and test it against normal, boundary and invalid cases.

billing.c

```
#include <stdio.h>
#include "tariff.h"

unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}

long compute_bill(unsigned int units)
{
   long paise = FIXED_CHARGE_PAISE;
   unsigned int u = units;
   if (u > SLAB2_UNITS) { paise += (long)(u - SLAB2_UNITS) * RATE3_PAISE; u = SLAB2_UNITS; }
   if (u > SLAB1_UNITS) { paise += (long)(u - SLAB1_UNITS) * RATE2_PAISE; u = SLAB1_UNITS; }
   paise += (long)u * RATE1_PAISE;
#ifdef DEBUG
   printf("[DEBUG] units=%u paise=%ld\n", units, paise);
#endif
   return paise;
}

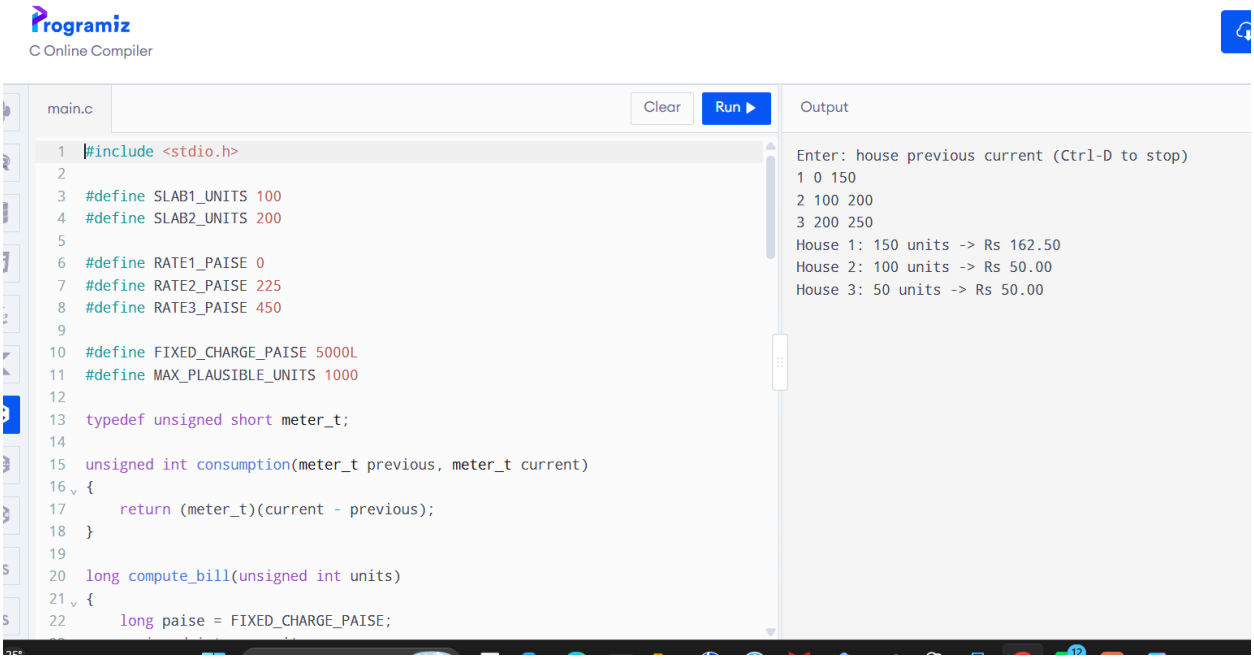
int main(void)
{
   int house, prev, curr;
   long total = 0;
   int n = 0;
   printf("Enter: house previous current (Ctrl-D to stop)\n");
   for (;;) {
      int r = scanf("%d %d %d", &house, &prev, &curr);
      if (r == EOF) break;
      if (r != 3) { printf("Invalid input (letters?) - skipping line\n"); int c; while ((c = getchar()) != '\n' && c != EOF); continue; }
      if (prev < 0 || curr < 0 || prev > 65535 || curr > 65535) { printf("House %d: reading out of range 0..65535\n", house);
continue; }
      unsigned int units = consumption((meter_t)prev, (meter_t)curr);
```

```
    if (curr < prev && units > MAX_PLAUSIBLE_UNITS) { printf("House %d: current < previous, not a plausible roll-over - rejected\n", house); continue; }
    long bill = compute_bill(units);
    total += bill; n++;
    printf("House %d: %u units -> Rs %ld.%02ld\n", house, units, bill/100, bill%100);
  }
  printf("Bills: %d  Total: Rs %ld.%02ld\n", n, total/100, total%100);
  return 0;
}
```

Test table

No.	Input (house prev curr)	Expected	Actual	Status
1	1 0 0 (zero units)	Rs 50.00	Rs 50.00	Pass
2	2 0 100 (boundary 100)	Rs 50.00	Rs 50.00	Pass
3	4 1000 1150 (150 units)	Rs 162.50	Rs 162.50	Pass
4	3 0 200 (boundary 200)	Rs 275.00	Rs 275.00	Pass
5	6 0 350	Rs 950.00	Rs 950.00	Pass
6	5 65400 120 (rollover)	256 units, Rs 527.00	256 units, Rs 527.00	Pass
7	7 abc 5 6 (letters)	Rejected	Invalid input – skipped	Pass
8	8 -5 10 (negative reading)	Rejected	Reading out of range	Pass
9	9 500 400 (current < previous, no rollover)	Rejected	Not a plausible roll-over – rejected	Pass
10	Five bills: 50, 150, 200, 250, 350 units	Hand total Rs 1937.50	Rs 1937.50	Pass

Hand calculation for test 10: 50 units → Rs 50.00; 150 → Rs 162.50; 200 → Rs 275.00; 250 → 50 + 100×2.25 + 50×4.50 = Rs 500.00; 350 → Rs 950.00. Sum = 50 + 162.50 + 275 + 500 + 950 = Rs 1937.50.



Alternative Concept – slab arrays and a loop

```
static const long limit[] = {100, 200, LONG_MAX};  /* upper limit of each slab */
static const long rate [] = {  0, 225, 450};
```

```
long paise = FIXED; long lower = 0;
for (int s = 0; s < 3 && units > lower; s++) {
   long top = units < limit[s] ? units : limit[s];
   paise += (top - lower) * rate[s];  lower = limit[s];
}
```

Time per bill: the if-version performs a fixed 2–3 tests, O(1) for the fixed tariff; the loop is O(s) for s slabs. Extra space: two arrays of s entries, O(s). The array design is easier to change – adding a slab is a data change, not a code change – and it is the stepping stone to reading the tariff from a file (Task 11).

Result: All 10 tests pass.